



LA INGENIERIA LEGAL _ CATEGORIAS DE LAS METRICAS _ CALIDAD DEL SOFTWARE

INGENIERÍA LEGAL EN SOFTWARE

El dúo entre los aspectos jurídicos y normativos contiene una serie relevante de aspectos que influyen de forma activa en el proceso fundamental a la hora de desarrollar software. Revisar la GUIA 1

Derechos de autor y licenciamiento: uso correcto de software libre, propietario o de código abierto.

Contratos y acuerdos: cláusulas de propiedad intelectual, confidencialidad, acuerdos de nivel de servicio (SLA). (Acuerdo de Nivel de Servicio, o Service Level Agreement)

Normatividad y estándares: cumplimiento de normas internacionales como ISO/IEC 12207 (procesos de ciclo de vida del software), ISO/IEC 25010 (calidad del producto), GDPR (protección de datos en Europa) o la Ley 1581 de 2012 en Colombia sobre habeas data.

Responsabilidad legal: en caso de fallos de seguridad, pérdidas de información o daños ocasionados por el software.

CATEGORIAS

Calidad del código : Referenciado a que miden la legibilidad, mantenibilidad y fiabilidad de su código base. Son importantes para mantener la eficiencia de tu código y reducir el tiempo de reparación.

Métricas de proceso : Referenciado a que ayudan a centrarse en la fluidez de su proceso de desarrollo de software. El tiempo de ciclo, el tiempo promedio dedicado a las historias de usuario y las métricas de producción identifican ineficiencias y le ayudan a optimizar los flujos de trabajo. Esta información le permite tomar decisiones basadas en datos y mantener sus proyectos en marcha.

Métricas del producto : Usar estas métricas puede ayudarle a evaluar la calidad de su software. Abarcan áreas como la funcionalidad, la usabilidad y el tiempo de actividad. Su seguimiento garantiza que su producto cumpla con las expectativas de los usuarios, ofrezca una experiencia fluida y reduzca las tasas de fallos.

Métricas ágiles : Estas métricas respaldan los principios ágiles y pueden ayudarte a supervisar el rendimiento del equipo y a adaptarte rápidamente. Métricas como el rendimiento, la velocidad y la satisfacción del cliente te ofrecen información práctica sobre cómo tu equipo aporta valor a lo largo del tiempo. También garantizan que tu desarrollo se alinee con los comentarios de los usuarios y las necesidades cambiantes.

COMPONENTES CLAVES DE UN SERVICIO SLA (ACUERDO DE NIVEL DE SERVICIO, O SERVICE LEVEL AGREEMENT)

Descripción de los servicios: Detalla los servicios específicos que se proporcionarán.

Métricas de rendimiento: Establece los indicadores clave (KPIs) para medir la calidad del servicio, como el tiempo de actividad o el tiempo de respuesta.

Responsabilidades: Define las obligaciones y responsabilidades tanto del proveedor como del cliente.

Reporte y monitoreo: Incluye cómo se rastreará el rendimiento del servicio y cómo se informará al cliente.

Exclusiones: Enumera los servicios o aspectos que no están cubiertos por el acuerdo.

Penalizaciones y resarcimientos: Especifica las consecuencias, como las penalizaciones por incumplimiento o los incentivos por superar las expectativas.

METRICAS DE LA CALIDAD DEL SOFTWARE

Referenciado hacia juicios y medidas utilizadas para evaluar qué tan bien se desarrolla una solución de software por un grupo. Son un compás que ayuda a entender la eficiencia, confiabilidad y rendimiento del software, asegurando que cumpla con los estándares y requisitos establecidos.

TIPOLOGIA Y CARACTERIZACION DE LAS METRICAS

MET1: Rotación total de códigos: Referenciado hacia la cantidad de líneas de código agregadas, modificadas o eliminadas de una base de código durante un período de tiempo.

Como se representa la métrica 1: l=línea, c=código y el subíndice respectivo (a: agregada, m: modificada y la e: eliminada).

La métrica 1 está conformada por los siguientes datos a saber:

Líneas de código agregadas = l_{ca}

Líneas de código modificadas = l_{cm}

Líneas de código eliminadas = l_{ce}

Periodo de tiempo de la medición (por lo general por Sprint (periodo de tiempo corto y fijo) = t

MET2: Tasa de fallas: Referenciado hacia la frecuencia con la que falla un producto de software.

Las posibles fallas pueden suceder por:

Lanzamientos apresurados: no hay tiempo suficiente para realizar un control de calidad adecuado antes de la fase de despliegue.

Recursos limitados: no hay suficientes horas/ojos para realizar un control de calidad adecuado.

Proceso de control de calidad inadecuado: desperfecto en el proceso real y aprobación del código por parte de pruebas.

La métrica 2 está conformada por los siguientes datos:

Esta métrica está formada por los siguientes datos:

Número de fallas = N_f

Periodo de tiempo de la medición (por lo general es por días) = t

$$\text{Tasa de fallas} = N_f / t$$

MET3: Disponibilidad del sistema: Referenciado hacia la cantidad de tiempo que un sistema está en funcionamiento. Compara el tiempo de actividad del sistema con su tiempo de inactividad durante un período de tiempo. Y considera que un sistema que está disponible es tan rápido como se espera y no falla a menos que ocurran condiciones imprevistas.

La métrica 3 está conformada por los siguientes datos:

Tiempo de actividad (medido en horas) = T_a

Tiempo de inactividad (medido en horas) = T_i

$$\text{Disponibilidad} = \frac{T_a}{T_a + T_i} * 100$$

MET4: Densidad de defectos: Referenciado hacia la cantidad de defectos en un producto de software en comparación con su tamaño. Esto significa que es un número relativo.

La métrica 4 a menudo se expresa como la cantidad de defectos por cada 1000 líneas de código.

La métrica 4 está formada por los siguientes datos:

Número de defectos = N_d

Número total de líneas de código = N_c

$$\text{Densidad de defectos} = \frac{N_d}{\frac{N_c}{1000}}$$



LA INGENIERIA LEGAL _ CATEGORIAS DE LAS METRICAS _ CALIDAD DEL SOFTWARE

UPEL

MET5: Tiempo medio de detección (MTTD): Referenciado hacia el tiempo promedio que le toma a tu equipo detectar errores en un producto de software.

La métrica 5 está formado por los siguientes datos:

- UPEL T_d Tiempo total para detectar todos los errores durante un período de tiempo = T_d
- UPEL N_e Número total de errores = N_e
- UPEL T_{d1} Tiempo para detectar un error = T_{d1}
- UPEL D_d Día y hora en que se detectó = D_d
- UPEL D_o Día y hora en que ocurrió = D_o

$$MTTD = \frac{T_d}{N_e}$$

UPEL

MET6: 6. Tiempo medio entre fallas (MTBF): Referenciado hacia el tiempo promedio entre dos fallas del sistema. Se trata de errores que se encuentran tras el lanzamiento del producto y que se deben, por ejemplo, a un defecto no detectado.

La métrica 6 está conformado por los siguientes datos:

- UPEL T_e Tiempo total de ejecución medido en horas= T_e
- UPEL N_f Número total de fallas = N_f

$$MTFB = \frac{T_e}{N_f}$$

UPEL

MET7: Tiempo medio de resolución (MTTR): Referenciado hacia el tiempo promedio que le toma a tu equipo resolver un error en un producto de software después de que alguien lo descubre.

La métrica 7 esta conformado por los siguientes datos:

$$MTTR = T_d/N_r$$

UPEL

MET8: Satisfacción del cliente (CSAT): Referenciado hacia un número que representa la forma en que los clientes experimentan su producto de software. Y se llega a ello recopilando y analizando los datos de las encuestas de satisfacción de tus clientes sobre el funcionamiento del software. No se incluyen datos sobre la calidad del servicio como por ejemplo el soporte en tu mesa de ayuda.

La métrica 8 esta conformado por los siguientes datos:

- UPEL C_s Número total de clientes satisfechos = C_s , siendo los clientes satisfechos aquellos que calificaron su satisfacción como “extremadamente satisfecho” y “satisfecho”.
- UPEL N_r Número total de respuestas a la encuesta de clientes = N_r

$$CSAT = C_s/N_r$$

MET9:Tiempo medio para remediar una vulnerabilidad: Referenciado hacia el tiempo promedio que le toma a tu equipo reparar las vulnerabilidades de ciberseguridad en tu producto de software.

La métrica 9 esta conformada por los siguientes datos:

- UPEL T_i Tiempo total entre el descubrimiento y la corrección de vulnerabilidades = T_i
- UPEL N_v Número total de vulnerabilidades = N_v
- UPEL t Periodo de tiempo = t , generalmente en días.

$$\text{Tiempo medio para remediar una vulnerabilidad} = T_i/N_v$$

UPEL

MET10: Cobertura del código: Referenciado hacia la evaluación de la cantidad de código fuente de tu producto para el cual existe una prueba unitaria.

Este indicador está formado por los siguientes datos:

- UPEL L_p Número de líneas de código probadas = L_p
- UPEL L_t Número total de líneas de código en el código base = L_t

$$\text{Cobertura del codigo} = L_p/L_t$$

**LA INGENIERIA LEGAL _ LAS PRUEBAS AL DESARROLLO DE SOFTWARE**

La ingeniería legal _desarrollo de software esta referenciada hacia la unión de los principios legales y éticos en el ciclo de vida del desarrollo para asegurar que el software cumple con leyes, regulaciones y estándares relevantes, mientras que las pruebas de software son un proceso fundamental para verificar la calidad, seguridad y funcionalidad del producto al identificar y corregir errores antes de su lanzamiento, mitigando así riesgos legales y operativos.

La relación entre Ingeniería Legal _ Pruebas de Software, está referenciado a que las PS son esenciales para la ingeniería legal porque ayudan a validar el cumplimiento de los requisitos de seguridad y funcionalidad, lo cual es un componente crítico del cumplimiento legal.

El cumplimiento legal en la ingeniería de software está referenciado a garantizar que el desarrollo, la distribución y el uso del software se adhieran a las leyes, regulaciones, estándares y principios éticos pertinentes.

CARACTERIZACION DE LAS PRUEBAS AL SOFTWARE

Reducen los costos de mantenimiento. A largo plazo, el realizar pruebas de software contribuye a que se requiera menos mantenimiento de la plataforma, lo que representa un ahorro para las empresas.

Previenen errores. Mediante las pruebas es posible **detectar fallas y corregirlas**. De esta manera, se ahorra tiempo y dinero, pues si el error se identifica luego de salir, se pueden afectar los procesos, se generan retrasos e inconformidades en los usuarios.

Mejoran la seguridad. Estas pruebas pueden ayudar a identificar y corregir vulnerabilidades de ciberseguridad en el software, lo que protege los datos tanto de la empresa como de los futuros usuarios.

Favorecen la experiencia del usuario. Ante un software que funciona correctamente y no presenta errores, **los usuarios se sienten más satisfechos**. De esta manera, mejora la imagen de la empresa, aumentan las ventas y se genera lealtad con la marca.

Aumentan la confiabilidad. Gracias a las pruebas de software, se logra un producto de calidad, confiable, seguro y menos propenso a fallar.

TIPOLOGIA DE LAS PRUEBAS**SEGÚN SU FUNCIONALIDAD**

Pruebas Funcionales: Se centran en probar el comportamiento de la aplicación, verificando que cada función cumpla con su propósito. Se basan en los requisitos y especificaciones del software.

Pruebas No Funcionales: Evalúan atributos como la **usabilidad**, la **seguridad**, el **rendimiento** y la **fiabilidad** del sistema. No se centran en el "qué hace", sino en el "cómo lo hace". Dentro de estas se incluyen:

PN Pruebas de Rendimiento: Miden la velocidad y la eficiencia de la aplicación bajo una carga de trabajo específica.

PN Pruebas de Estrés: Determinan la capacidad de un sistema para manejar cargas de trabajo extremas o situaciones de estrés.

PN Pruebas de Usabilidad: Valoran la facilidad con la que los usuarios pueden usar la aplicación.

PN Pruebas de Seguridad: Identifican vulnerabilidades y aseguran que la información esté protegida contra accesos no autorizados.

SEGÚN EL CAMBIO

Pruebas de Regresión: Referenciado hacia que se ejecutan para verificar que los nuevos cambios o correcciones en el código no hayan introducido nuevos errores o afectado negativamente las funcionalidades existentes.

Pruebas de Humo (Smoke Testing): Referenciado hacia que son un conjunto rápido de pruebas que verifican la estabilidad de la compilación de software para decidir si es viable continuar con pruebas más detalladas.

SEGÚN EL NIVEL DE DESARROLLO

Pruebas de Aceptación: Referenciado a que son las pruebas finales antes de la entrega del producto. Confirman que el software satisface las necesidades y requisitos del usuario final y del negocio. Estas pueden ser realizadas por el cliente (UAT - User Acceptance Testing) o por el equipo interno (Alpha/Beta testing).

Pruebas de Integración: Referenciado a que verifican que los módulos del software interactúen y se comuniquen correctamente entre sí. El objetivo es encontrar fallos en la interfaz entre los componentes.

Pruebas de Sistema: Referenciado a que evalúan el sistema de software completo y totalmente integrado. Aquí se verifica que el sistema cumpla con los requisitos funcionales y no funcionales de principio a fin.

Pruebas Unitarias: Referenciado a que se realizan en los componentes o módulos más pequeños y aislados del código (como una función o clase) para verificar que funcionan correctamente. Generalmente, las hacen los propios desarrolladores.

SEGÚN SU ENFOQUE

Pruebas de Caja Negra (Black Box Testing): Referenciado a que se centran en las funcionalidades del software sin conocer su estructura interna (código, diseño). Se simulan las interacciones del usuario.

Pruebas de Caja Gris (Gray Box Testing): Referenciado a que combinan elementos de las pruebas de caja negra y blanca. Se tiene un conocimiento parcial de la estructura interna.

Pruebas de Caja Blanca (White Box Testing): Referenciado a que requieren conocimiento del código interno de la aplicación. Se analizan los caminos del código, bucles y estructuras de datos para asegurar su correcto funcionamiento.



LA INGENIERIA LEGAL _ EL SOFTWARE

DE ACUERDO CON LA RELACION CON EL USUARIO

- Interactivo: Referenciado a que requiere constante interacción (juegos, editores de texto).
- De procesamiento por lotes: Referenciado a que procesa grandes volúmenes sin interacción constante (programas de nómina, análisis de datos).
- En tiempo real: Referenciado a que responde de inmediato a eventos (sistemas de control aéreo, marcapasos, cajeros automáticos).

DE ACUERDO A SU PROPOSITO

- Software de sistema:** Referenciado hacia que administra los recursos del hardware y sirve de base para otros programas.
- TENGA PRESENTE:** sistemas operativos (Windows, Linux, macOS), drivers, utilidades.
- Software de aplicación:** Referenciado hacia que está diseñado para que el usuario realice tareas específicas.
- TENGA PRESENTE:** Word, Excel, navegadores web, sistemas contables, apps móviles.
- Software de desarrollo (herramientas de programación):** Referenciado hacia que facilitan la creación, prueba y mantenimiento de software.
- TENGA PRESENTE:** compiladores, intérpretes, editores de código, IDEs (Eclipse, Visual Studio).

DE ACUERDO A SU DISTRIBUCION O ACCESO

- Software shareware: Se distribuye gratis por un tiempo, luego exige pago.
- Software propietario: Tiene dueño, con licencias restrictivas (Windows, Office).
- Software libre: Libertad de uso, copia, modificación y distribución (Linux, LibreOffice).
- Software freeware: Gratis, pero sin acceso al código fuente.
- Software de código abierto (Open Source): El código está disponible, aunque no siempre es libre.
- Software comercial: Se vende con fines de lucro.

DE ACUERDO A SU FORMA DE USO

- Software de escritorio: Referenciado hacia que se instala localmente (Photoshop, AutoCAD).
- Software web: Referenciado hacia que es accedido desde un navegador (Google Docs, Gmail).
- Software en la nube (SaaS): Referenciado hacia que funciona desde servidores remotos (Dropbox, Office 365).
- Software embebido: Referenciado hacia que corre dentro de dispositivos electrónicos (firmware de un televisor, un Router o un carro).
- Software móvil: Referenciado hacia que está diseñado para smartphones o tabletas (WhatsApp, Uber).

DE ACUERDO A SU NIVEL DE APLICACIÓN DE INGENIERIA

- Software de sistemas operativos
- Software de redes y comunicaciones
- Software científico y de ingeniería
- Software empresarial
- Software de inteligencia artificial (sistemas expertos, machine learning, asistentes virtuales).
- Software de entretenimiento (juegos, multimedia).
- Software educativo (plataformas e-learning, simuladores).

DE ACUERDO A LA ARQUITECTURA Y TECNOLOGIA

- Microservicios: Referenciado hacia la Arquitectura modular utilizada por Amazon, Netflix.
- Software basado en inteligencia artificial (AI-Driven): ChatGPT, Copilot, MidJourney.
- Aplicaciones en contenedores: Docker, Kubernetes.
- Software serverless (sin servidor): AWS Lambda, Google Cloud Functions.
- Aplicaciones de realidad aumentada (AR) y realidad virtual (VR): Oculus, Pokémon Go.
- Software de blockchain y Web3: MetaMask, OpenSea.



LA INENIERIA LEGAL_ TECNOLOGIAS DE DESARROLLO DE SOFTWARE

Python: Referenciado hacia el lenguaje de programación de alto nivel, posee simplicidad y versatilidad. Es una excelente opción para principiantes ya que tiene una sintaxis fácil de aprender y es ampliamente utilizada en la comunidad científica.

Se utiliza para el desarrollo web, el análisis de datos, la inteligencia artificial y el aprendizaje automático. Las sólidas bibliotecas del lenguaje lo convierten en una poderosa herramienta en el desarrollo de software.



JavaScript: Referenciado hacia el lenguaje de programación más utilizado en el mundo y es esencial en el desarrollo web. Su versatilidad permite a los desarrolladores crear páginas web interactivas y aplicaciones web.

Los marcos de JavaScript, como React y Vue, han facilitado a los desarrolladores la creación de aplicaciones web complejas con facilidad.



Java: Referenciado hacia el lenguaje de programación popular utilizado en el desarrollo de nivel empresarial. Es conocido por su portabilidad y puede ejecutarse en varios sistemas operativos, lo que lo convierte en una excelente opción para aplicaciones multiplataforma. Java se utiliza para desarrollar aplicaciones de escritorio, aplicaciones web y aplicaciones móviles Android.



AngularJS: Referenciado hacia que es un marco de JavaScript utilizado para crear aplicaciones web dinámicas. Es un marco de código abierto que proporciona una estructura para crear aplicaciones web complejas. AngularJS tiene un poderoso conjunto de herramientas que facilita a los desarrolladores la creación de aplicaciones escalables y mantenibles.



PHP: Referenciado hacia un lenguaje de secuencias de comandos del lado del servidor utilizado para el desarrollo web. Es un lenguaje de código abierto que se usa ampliamente para desarrollar sitios web dinámicos y aplicaciones web. Los marcos PHP, como Laravel y Symfony, han facilitado a los desarrolladores la creación rápida de aplicaciones web complejas.



SQL: Referenciado hacia (lenguaje de consulta estructurado) es un lenguaje de gestión de bases de datos que se utiliza para almacenar y recuperar datos en una base de datos relacional. Es un lenguaje estándar utilizado en la gestión de datos y es esencial en el desarrollo de software. SQL se utiliza para la gestión de bases de datos, el análisis de datos y la generación de informes de datos.



C#: Referenciado el lenguaje de programación desarrollado por Microsoft para desarrollar aplicaciones y juegos de Windows. Es un lenguaje orientado a objetos que es fácil de aprender y se usa ampliamente en la industria del desarrollo de juegos. C# tiene un sólido conjunto de bibliotecas y marcos que facilitan a los desarrolladores la creación de aplicaciones complejas.



Swift: Referenciado hacia el lenguaje de programación desarrollado por Apple para desarrollar aplicaciones iOS y macOS. Es un lenguaje fácil de aprender que ha ganado popularidad debido a su velocidad y seguridad. Swift tiene un conjunto de herramientas poderosas que facilitan a los desarrolladores la creación de aplicaciones complejas.



Kotlin: Referenciado hacia el lenguaje de programación desarrollado por JetBrains para desarrollar aplicaciones Android. Es un lenguaje moderno que es fácil de aprender y es una excelente opción para el desarrollo de Android. Kotlin tiene un conjunto de herramientas poderosas que facilitan a los desarrolladores la creación de aplicaciones complejas.



INVESTIGAR

[Ruby](#), [Go](#), [Rust](#) y [TypeScript](#).



ACTIVIDAD GRUPAL : Analice y diseñe de forma clara y precisa todos los elementos contenidos en los LA INGENIERIA LEGAL _ CATEGORIAS DE LAS METRICAS _ CALIDAD DEL SOFTWARE, LA INGENIERIA LEGAL _ LAS PRUBAS AL DESARROLLO DE SOFTWARE_ LA INGENIERIA LEGAL _ EL SOFTWARE_ LA INENIERIA LEGAL_ TECNOLOGIAS DE DESARROLLO DE SOFTWARE.

FORMA DE ENTREGA DEL DOCUMENTO POR CIPAS

TODO EN MAYUSCULA

GUIA1_INGLEGALGRUPO2_APELLIDOS Y NOMBRE

DECADA UNO DE LOS INTEGRANTES DEL CIPA